

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет: Инженерно-экономический

Кафедра: Экономической информатики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
на тему:

Выполнил

Филяковский С. В.
гр. 377901

Проверила

Семенюк А. В.

Минск 2026

Вариант 12

Построение сценария распределения задач между несколькими исполнителями

Требуется:

- создать таблицу очереди задач;
- открыть несколько сессий;
- в каждой сессии извлекать следующую задачу;
- использовать `FOR UPDATE SKIP LOCKED`;
- доказать, что задача не берётся двумя исполнителями одновременно;
- сравнить сценарий с обычным `FOR UPDATE`.

Пример элемента

```
SELECT * FROM jobs
WHERE status = 'new'
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

docker-compose

```
services:
  postgres:
    image: postgres:16
    container_name: postgres_task_queue
    restart: no
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data/
      - ./init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
      - ./seed.sql:/docker-entrypoint-initdb.d/02-seed.sql:ro
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: task_queue_db
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

init

```
-- =====  
  
-- Создание таблицы очереди задач  
-- =====  
  
CREATE TABLE IF NOT EXISTS jobs (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  description TEXT,  
  status VARCHAR(20) DEFAULT 'new', -- new / processing / completed  
  priority INT DEFAULT 0,  
  worker_id INT,  
  created_at TIMESTAMP DEFAULT now(),  
  started_at TIMESTAMP,  
  completed_at TIMESTAMP  
);  
  
-- Индексы для ускорения поиска задач  
CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status);  
CREATE INDEX IF NOT EXISTS idx_jobs_status_priority ON jobs(status, priority DESC);  
CREATE INDEX IF NOT EXISTS idx_jobs_worker_id ON jobs(worker_id);  
  
-- Таблица для логирования обработки задач  
CREATE TABLE IF NOT EXISTS job_log (  
  id SERIAL PRIMARY KEY,  
  job_id INT REFERENCES jobs(id),  
  worker_id INT NOT NULL,  
  action VARCHAR(50), -- started / completed  
  action_time TIMESTAMP DEFAULT now()  
);  
  
-- Индекс для логирования  
CREATE INDEX IF NOT EXISTS idx_job_log_job_id ON job_log(job_id);  
  
SELECT 'Tables created successfully' AS status;
```

seed

```
-- =====
```

```
-- Генератор тестовых задач для очереди
```

```
-- =====
```

```
TRUNCATE job_log, jobs RESTART IDENTITY CASCADE;
```

```
-- Создаем 1000 задач с разным приоритетом
```

```
INSERT INTO jobs (title, description, status, priority)
```

```
SELECT
```

```
'Task ' || i,
```

```
'Description for task ' || i || ' - ' || md5(random()::text),
```

```
'new',
```

```
(random() * 10)::int -- приоритет от 0 до 10
```

```
FROM generate_series(1, 1000) AS i;
```

```
SELECT 'jobs' AS table_name, count(*) FROM jobs;
```

analyze

```
--
=====

-- АНАЛИЗ РАСПРЕДЕЛЕНИЯ ЗАДАЧ МЕЖДУ ИСПОЛНИТЕЛЯМИ
-- Используем FOR UPDATE SKIP LOCKED
=====

SET track_io_timing = on;

=====
-- ЭТАП 1: БАЗОВЫЙ СЦЕНАРИЙ - FOR UPDATE SKIP LOCKED
=====
\echo '=====
\echo 'ЭТАП 1: ДЕМОНСТРАЦИЯ FOR UPDATE SKIP LOCKED'
\echo '=====

-- Проверяем количество новых задач
SELECT 'Total new jobs:' as info, count(*) FROM jobs WHERE status = 'new';

-- Пример извлечения одной задачи с блокировкой
\echo 'Извлекаем задачу с FOR UPDATE SKIP LOCKED:'
BEGIN;
SELECT * FROM jobs
WHERE status = 'new'
ORDER BY priority DESC, created_at ASC
FOR UPDATE SKIP LOCKED
LIMIT 1;
-- Пока транзакция открыта, другие сессии пропустят эту строку
ROLLBACK;

=====
-- ЭТАП 2: СИМУЛЯЦИЯ ПАРАЛЛЕЛЬНОЙ ОБРАБОТКИ (2 рабочих)
=====
\echo '=====
\echo 'ЭТАП 2: ПРОВЕРКА ОТСУТСТВИЯ КОНФЛИКТОВ'
\echo '=====

-- Сбрасываем статусы
UPDATE jobs SET status = 'new', worker_id = NULL, started_at = NULL;

-- Создаем функцию для имитации работы воркера
CREATE OR REPLACE FUNCTION process_next_job(p_worker_id INT)
RETURNS TABLE(job_id INT, job_title VARCHAR) AS $$
DECLARE
v_locked_job RECORD;
BEGIN
```

```

-- Извлекаем следующую доступную задачу
SELECT j.id, j.title INTO v_locked_job
FROM jobs j
WHERE j.status = 'new'
ORDER BY j.priority DESC, j.created_at ASC
FOR UPDATE SKIP LOCKED
LIMIT 1;
IF v_locked_job.id IS NOT NULL THEN
-- Обновляем статус задачи
UPDATE jobs
SET status = 'processing',
worker_id = p_worker_id,
started_at = now()
WHERE id = v_locked_job.id;
-- Логгируем начало обработки
INSERT INTO job_log (job_id, worker_id, action)
VALUES (v_locked_job.id, p_worker_id, 'started');
-- Имитируем обработку (в реальности здесь была бы работа)
PERFORM pg_sleep(0.01);
-- Завершаем задачу
UPDATE jobs
SET status = 'completed',
completed_at = now()
WHERE id = v_locked_job.id;
-- Логгируем завершение
INSERT INTO job_log (job_id, worker_id, action)
VALUES (v_locked_job.id, p_worker_id, 'completed');
job_id := v_locked_job.id;
job_title := v_locked_job.title;
RETURN NEXT;
END IF;
RETURN;
END;
$$ LANGUAGE plpgsql;

```

```
\echo 'Функция process_next_job создана'
```

```

-- =====
-- ЭТАП 3: ТЕСТИРОВАНИЕ ПАРАЛЛЕЛЬНОЙ ОБРАБОТКИ
-- =====
\echo '=====
\echo 'ЭТАП 3: ЗАПУСК 2-Х "ПАРАЛЛЕЛЬНЫХ" РАБОЧИХ'
\echo '=====

```

```

-- Сброс
UPDATE jobs SET status = 'new', worker_id = NULL, started_at = NULL, completed_at = NULL;
TRUNCATE job_log;

```

```

-- Симуляция работы двух воркеров (последовательно, но с проверкой)
\echo 'Worker 1 берет задачу:'

```

```
SELECT * FROM process_next_job(1);
```

```
\echo 'Worker 2 берет задачу (должна быть ДРУГАЯ):'
```

```
SELECT * FROM process_next_job(2);
```

```
\echo 'Worker 1 берет еще одну задачу:'
```

```
SELECT * FROM process_next_job(1);
```

```
-- Проверяем, что задачи разные
```

```
\echo 'Проверка распределения задач между воркерами:'
```

```
SELECT  
worker_id,  
count(*) as tasks_processed,  
array_agg(job_id ORDER BY id) as job_ids  
FROM job_log  
WHERE action = 'started'  
GROUP BY worker_id  
ORDER BY worker_id;
```

```
-- =====  
-- ЭТАП 4: СРАВНЕНИЕ С ОБЫЧНЫМ UPDATE (БЕЗ SKIP LOCKED)  
-- =====  
  
\echo '====='  
\echo 'ЭТАП 4: СРАВНЕНИЕ С FOR UPDATE (БЕЗ SKIP LOCKED)'  
\echo '====='
```

```
-- Сброс
```

```
UPDATE jobs SET status = 'new', worker_id = NULL;  
TRUNCATE job_log;
```

```
-- Функция с обычным FOR UPDATE (блокирующая)
```

```
CREATE OR REPLACE FUNCTION process_next_job_blocking(p_worker_id INT)  
RETURNS TABLE(job_id INT, job_title VARCHAR) AS $$  
DECLARE  
v_locked_job RECORD;  
BEGIN  
-- Блокирующий вариант - ждет освобождения строки  
SELECT j.id, j.title INTO v_locked_job  
FROM jobs j  
WHERE j.status = 'new'  
ORDER BY j.priority DESC, j.created_at ASC  
FOR UPDATE -- БЕЗ SKIP LOCKED - будет ждать!  
LIMIT 1;  
IF v_locked_job.id IS NOT NULL THEN  
UPDATE jobs  
SET status = 'processing',  
worker_id = p_worker_id,  
started_at = now()  
WHERE id = v_locked_job.id;  
INSERT INTO job_log (job_id, worker_id, action)
```

```

VALUES (v_locked_job.id, p_worker_id, 'started');
PERFORM pg_sleep(0.01);
UPDATE jobs
SET status = 'completed',
    completed_at = now()
WHERE id = v_locked_job.id;
INSERT INTO job_log (job_id, worker_id, action)
VALUES (v_locked_job.id, p_worker_id, 'completed');
job_id := v_locked_job.id;
job_title := v_locked_job.title;
RETURN NEXT;
END IF;
RETURN;
END;
$$ LANGUAGE plpgsql;

```

```
\echo 'Функция process_next_job_blocking создана'
```

```

=====
-- ЭТАП 5: MASS PROCESSING - ОБРАБОТКА ВСЕХ ЗАДАЧ
=====
\echo '=====
\echo 'ЭТАП 5: ОБРАБОТКА 100 ЗАДАЧ ДВУМЯ МЕТОДАМИ'
\echo '=====

```

```

-- Метод 1: SKIP LOCKED
\echo '--- МЕТОД 1: FOR UPDATE SKIP LOCKED ---'

```

```

-- Сброс
UPDATE jobs SET status = 'new', worker_id = NULL, started_at = NULL, completed_at = NULL;
TRUNCATE job_log;

```

```

-- Создаем процедуру для массовой обработки
CREATE OR REPLACE PROCEDURE mass_process_jobs_skip_locked(
    p_worker_count INT,
    p_tasks_per_worker INT
) AS $$
DECLARE
    v_worker_id INT;
    v_task_num INT;
    v_job_record RECORD;
BEGIN
    FOR v_worker_id IN 1..p_worker_count LOOP
        FOR v_task_num IN 1..p_tasks_per_worker LOOP
            -- Извлекаем и обрабатываем задачу
            SELECT j.id, j.title INTO v_job_record
            FROM jobs j
            WHERE j.status = 'new'
            ORDER BY j.priority DESC, j.created_at ASC
            FOR UPDATE SKIP LOCKED

```

```

LIMIT 1;
IF v_job_record.id IS NOT NULL THEN
UPDATE jobs
SET status = 'processing',
worker_id = v_worker_id,
started_at = now()
WHERE id = v_job_record.id;
INSERT INTO job_log (job_id, worker_id, action)
VALUES (v_job_record.id, v_worker_id, 'started');
PERFORM pg_sleep(0.001); -- имитация работы
UPDATE jobs
SET status = 'completed',
completed_at = now()
WHERE id = v_job_record.id;
INSERT INTO job_log (job_id, worker_id, action)
VALUES (v_job_record.id, v_worker_id, 'completed');
END IF;
END LOOP;
END LOOP;
END;
$$ LANGUAGE plpgsql;

```

```

\echo 'Запускаем обработку 100 задач 2 воркерами (SKIP LOCKED)...'
\timing on
CALL mass_process_jobs_skip_locked(2, 50);
\timing off

```

```

\echo 'Результаты:'
SELECT
worker_id,
count(*) as tasks_completed
FROM job_log
WHERE action = 'completed'
GROUP BY worker_id
ORDER BY worker_id;

```

```

\echo 'Всего обработано:'
SELECT count(*) as total_completed FROM jobs WHERE status = 'completed';

```

```

-- =====
-- ЭТАП 6: АНАЛИЗ БЛОКИРОВОК И ПРОИЗВОДИТЕЛЬНОСТИ
-- =====
\echo '=====
\echo 'ЭТАП 6: АНАЛИЗ ЭФФЕКТИВНОСТИ SKIP LOCKED'
\echo '=====

```

```

-- Проверяем отсутствие дублирования
\echo 'Проверка на дублирование (должно быть 0):'
SELECT
job_id,

```

```
count(*) as process_count
FROM job_log
WHERE action = 'started'
GROUP BY job_id
HAVING count(*) > 1;
```

```
-- Статистика по задачам
\echo 'Статистика обработки:'
SELECT
status,
count(*) as count,
COALESCE(avg(EXTRACT(EPOCH FROM (completed_at - started_at))), 0) as
avg_processing_time_sec
FROM jobs
GROUP BY status;
```

```
-- =====
-- ЭТАП 7: EXPLAIN ANALYZE для SKIP LOCKED
-- =====
```

```
\echo '===== '
\echo 'ЭТАП 7: EXPLAIN ANALYZE - SKIP LOCKED'
\echo '===== '
```

```
-- Сброс для чистого теста
UPDATE jobs SET status = 'new', worker_id = NULL;
```

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT * FROM jobs
WHERE status = 'new'
ORDER BY priority DESC, created_at ASC
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

```
-- =====
-- ЭТАП 8: СРАВНЕНИЕ ПЛАНОВ ВЫПОЛНЕНИЯ
-- =====
```

```
\echo '===== '
\echo 'ЭТАП 8: FOR UPDATE (без SKIP LOCKED) - для сравнения'
\echo '===== '
```

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT * FROM jobs
WHERE status = 'new'
ORDER BY priority DESC, created_at ASC
FOR UPDATE
LIMIT 1;
```

```
-- =====
-- ЭТАП 9: ИТОГОВАЯ СТАТИСТИКА
-- =====
```

```

\echo '=====
\echo 'ИТОГОВАЯ СТАТИСТИКА'
\echo '=====

SELECT
'Общее количество задач' as metric,
count(*)::text as value
FROM jobs
UNION ALL
SELECT 'Выполнено задач', count(*)::text FROM jobs WHERE status = 'completed'
UNION ALL
SELECT 'Записей в логe', count(*)::text FROM job_log
UNION ALL
SELECT 'Уникальных воркеров', count(DISTINCT worker_id)::text FROM job_log;

\echo '=====
\echo 'ВЫВОДЫ:'
\echo '1. FOR UPDATE SKIP LOCKED позволяет избежать блокировок'
\echo '2. Каждая задача обрабатывается ТОЛЬКО ОДНИМ воркером'
\echo '3. Воркеры не ждут освобождения строк - высокая параллельность'
\echo '4. Идеально для очередей задач и фоновых джоб'
\echo '=====

```

Результаты

```

=====
ЭТАП 1: ДЕМОНСТРАЦИЯ FOR UPDATE SKIP LOCKED
=====
      info      | count
-----+-----
Total new jobs: | 1000
(1 row)

Извлекаем задачу с FOR UPDATE SKIP LOCKED:
BEGIN
id | title | description | status | priority | worker_id | created_at | started_at | completed_at
-----+-----+-----+-----+-----+-----+-----+-----+-----
1 | Task 1 | Description for task 1 - e3cb7903e06c3e714d322fdb177b9a4 | new | 10 | | 2026-09-16 20:32:34.510063 | | 
(1 row)

ROLLBACK
=====
ЭТАП 2: ПРОВЕРКА ОТСУТСТВИЯ КОНФЛИКТОВ
=====
UPDATE 1000
CREATE FUNCTION
Функция process_next_job создана
=====

```

```

=====
ЭТАП 3: ЗАПУСК 2-Х "ПАРАЛЛЕЛЬНЫХ" РАБОЧИХ
=====
UPDATE 1000
TRUNCATE TABLE
Worker 1 берет задачу:
 job_id | job_title
-----+-----
      1 | Task 1
(1 row)

Worker 2 берет задачу (должна быть ДРУГАЯ):
 job_id | job_title
-----+-----
      3 | Task 3
(1 row)

Worker 1 берет еще одну задачу:
 job_id | job_title
-----+-----
     33 | Task 33
(1 row)

Проверка распределения задач между воркерами:
worker_id | tasks_processed | job_ids
-----+-----+-----
      1 |      2 | {1,33}
      2 |      1 | {3}
(2 rows)

```

```
=====
ЭТАП 8: FOR UPDATE (без SKIP LOCKED) - для сравнения
=====

QUERY PLAN
-----
Limit (cost=4.88..8.86 rows=1 width=550) (actual time=0.088..0.089 rows=1 loops=1)
  Buffers: shared hit=104
  -> LockRows (cost=4.88..32.79 rows=7 width=550) (actual time=0.088..0.088 rows=1 loops=1)
    Buffers: shared hit=104
    -> Incremental Sort (cost=4.88..32.72 rows=7 width=550) (actual time=0.084..0.084 rows=1 loops=1)
      Sort Key: priority DESC, created_at
      Presorted Key: priority
      Full-sort Groups: 1 Sort Method: quicksort Average Memory: 33kB Peak Memory: 33kB
      Buffers: shared hit=103
      -> Index Scan using idx_jobs_status_priority on jobs (cost=0.28..32.40 rows=7 width=550) (actual time=0.018..0.065 rows=58 loops=1)
        Index Cond: ((status)::text = 'new'::text)
        Buffers: shared hit=103

Planning Time: 0.054 ms
Execution Time: 0.287 ms
(14 rows)

=====
ИТОГОВАЯ СТАТИСТИКА
=====
metric | value
-----+-----
Общее количество задач | 1000
Выполнено задач | 0
Записей в логе | 200
Уникальных воркеров | 2
(4 rows)

=====
ВЫВОДЫ:
1. FOR UPDATE SKIP LOCKED позволяет избежать блокировок
2. Каждая задача обрабатывается ТОЛЬКО ОДНИМ воркером
3. Воркеры не ждут освобождения строк - высокая параллельность
4. Идеально для очередей задач и фоновых джоб
-----
1 | 50
2 | 50
(2 rows)

Всего обработано:
total_completed
-----
100
(1 row)
```

```
=====
ЭТАП 6: АНАЛИЗ ЭФФЕКТИВНОСТИ SKIP LOCKED
=====
Проверка на дублирование (должно быть 0):
job_id | process_count
-----+-----
(0 rows)

Статистика обработки:
status | count | avg_processing_time_sec
-----+-----+-----
completed | 100 | 0.000000000000000000
new | 900 | 0
(2 rows)

=====
ЭТАП 7: EXPLAIN ANALYZE - SKIP LOCKED
=====
UPDATE 1000

QUERY PLAN
-----
Limit (cost=4.88..8.86 rows=1 width=550) (actual time=0.227..0.227 rows=1 loops=1)
  Buffers: shared hit=104
  -> LockRows (cost=4.88..32.79 rows=7 width=550) (actual time=0.225..0.226 rows=1 loops=1)
    Buffers: shared hit=104
    -> Incremental Sort (cost=4.88..32.72 rows=7 width=550) (actual time=0.220..0.221 rows=1 loops=1)
      Sort Key: priority DESC, created_at
      Presorted Key: priority
      Full-sort Groups: 1 Sort Method: quicksort Average Memory: 33kB Peak Memory: 33kB
      Buffers: shared hit=103
      -> Index Scan using idx_jobs_status_priority on jobs (cost=0.28..32.40 rows=7 width=550) (actual time=0.060..0.186 rows=58 loops=1)
        Index Cond: ((status)::text = 'new'::text)
        Buffers: shared hit=103

Planning Time: 0.091 ms
Execution Time: 0.433 ms
(14 rows)
```